

Project Report

- **Please list out changes in the directions of your project if the final project is different from your original proposal (based on your stage 1 proposal submission).**
 - Exclusion of Weather and Ski Resort Rental Details: In the proposal, weather updates, food and beverage costs, and equipment rental prices were listed as key functionalities to help users plan their ski trips comprehensively. These aspects were not included in the final project.
 - Nearby Restaurant Information: The proposal also included providing information about restaurants near ski resorts to enhance the user experience. This feature was omitted in the final project.
 - Lessons Functionality: A lesson feature was implemented, allowing users to select preferred lessons at the ski resorts and provide ratings for the instructors. This functionality was not part of the original proposal and represents a significant enhancement, focusing on user interaction and feedback.
- **Discuss what you think your application achieved or failed to achieve regarding its usefulness.**
 - We have developed an application to help users discover ski resorts in Europe. Key features include filtering ski resorts based on user-defined criteria, adding comments about resorts and instructors, and registering for ski lessons. However, we were unable to implement food recommendations due to the lack of a suitable dataset.
- **Discuss if you change the schema or source of the data for your application**
 - Yes, we changed the schema for our application to better align with the functional requirements.
 - First, we converted all ID fields from strings to integers, enabling automatic ID incrementation when inserting new rows of data.
 - To enforce limits on course registrations, we added a new attribute, MaxStudent, to the Lesson table. This ensures that each course has a defined maximum capacity, and we implemented logic to count the registered users dynamically using the Take table. These changes collectively enhanced the schema's scalability, clarity, and alignment with the application's functional requirements.

- **Discuss what you change to your ER diagram and/or your table implementations. What are some differences between the original design and the final design? Why? What do you think is a more suitable design?**
 - We introduced a new relational table named Take to establish a direct relationship between users and lessons. This table records which user is registered for which lesson, simplifying queries related to course registrations and providing a clearer relational structure.
 - Since the relationship between users and lessons is many-to-many, adding a new table should be the most suitable design.
- **Discuss what functionalities you added or removed. Why?**
 - Functionalities Added:
 - Course Functionality:
 - Description: Visitors can now select lessons at the ski resorts and rate the instructors. This feature allows for enhanced user interaction and provides feedback on the quality of instruction.
 - Reason for Addition: This was added to increase user engagement and to provide an interactive experience, allowing visitors to make more informed decisions when choosing lessons.
 - Functionalities Removed:
 - Weather Information:
 - Description: The original proposal included the functionality to provide real-time weather updates for ski resorts.
 - Reason for Removal: Likely removed to simplify the project scope and focus on core features, as implementing real-time weather updates requires integration with external APIs and significant additional effort.
 - Food, Beverage, and Equipment Rental Information:
 - Description: The project initially planned to include details about food, beverages, and rental equipment prices at the resorts.
 - Reason for Removal: These features might have been deprioritized due to the complexity of gathering such data or to streamline the development process by focusing on more critical functionalities.
 - Nearby Restaurant Information:

- Description: The original proposal aimed to list nearby restaurants to enhance the visitor experience.
 - Reason for Removal: This feature was likely removed to avoid overwhelming the project with non-essential features and to focus resources on implementing the course and instructor rating functionalities.
- **Explain how you think your advanced database programs complement your application.**
 - Our advanced queries in the AdvancedResortQueries procedure deliver tailored, high-performance data retrieval to meet diverse user needs, enhancing web platform's functionality:

(Query1) Intermediate Slopes : Retrieves resorts with more than 10 intermediate slopes and above-average snowfall, ranked by slope count, serving users seeking balanced skiing experiences.

(Query2) Beginner Slopes: Focuses on resorts with over 10 beginner slopes and above-average snowfall, ideal for novice skiers and families prioritizing safety and ease.

(Query3) Difficult Slopes: Targets resorts with over 10 difficult slopes and above-average snowfall, ranked by difficulty, catering to expert skiers and thrill-seekers.

Each query ensures precise, curated results by combining slope difficulty and snowfall data, with a top 15 limit for relevance and efficiency. This dynamic, server-side processing minimizes response times, reduces client-server overhead, and delivers a seamless, personalized user experience. By addressing distinct user preferences, the queries make our platform more engaging, responsive, and adaptable for a wide range of audiences.
- **Each team member should describe one technical challenge that the team encountered. This should be sufficiently detailed such that another future team could use this as helpful advice if they were to start a similar project or where to maintain your project.**
 - Ziqi Liu: Designing relationships and maintaining normalization in the database to ensure data consistency while minimizing redundancy. One specific challenge was implementing the InstructorComments and ResortComments tables to properly calculate and update average ratings dynamically. This required the use of triggers for INSERT, UPDATE, and DELETE operations, which occasionally led to unexpected behaviors, such as incorrect calculations when all comments were deleted or when new ratings were added.

- Jing Li: Implementing dynamic triggers for rating updates was particularly challenging because of edge cases like deleting the last comment or recalculating averages when comments were updated or removed. These scenarios required careful planning to avoid null or incorrect values in the AverageRating field for instructors and resorts. Another issue was ensuring compatibility when additional features like Favorites were added, which increased database complexity.
 - Yutong Chen: During the development process, I faced challenges such as ensuring data consistency across multiple tables, especially when updating or retrieving dependent values like RegisteredUsers. Handling transactions to maintain atomicity and prevent partial updates required careful SQL design and debugging. Debugging intermittent errors like "Unread result found" further complicated the process, as they required identifying and properly closing open database cursors. These challenges highlighted the importance of thorough schema design, proper error handling, and iterative testing.
 - Yu-Tao Sun: Developing a responsive and user-friendly front end and implement several APIs to get data required in our application to dynamically display complex data, such as real-time lesson availability and updated average ratings for resorts and instructors. A specific issue was handling errors when the backend returned unexpected data or when the database schema changed.
- **Are there other things that changed comparing the final application with the original proposal?**
 - Removed Features:
 - Weather Integration: The final application does not include real-time weather data for resorts, which was originally proposed.
 - Food and Beverage Information: The initial plan included information about dining options and chemical supplies rentals near resorts, which were excluded from the final design.
 - Nearby Restaurants: The final application does not display information about restaurants close to the resorts, as initially proposed.
 - Added Features:
 - Lesson and Instructor System: A new feature was added, allowing users to browse lessons, select instructors, and leave ratings and comments for them. This was not in the original plan.
 - User Interaction with Comments: The application expanded user functionality by enabling them to leave comments and ratings for resorts and instructors, enriching user engagement.

- Advanced Filters: Additional filtering options for lessons and resorts were introduced to improve search capabilities.
- Database Design Changes:
 - New table Take was added to support the lesson and instructor features.
 - Existing tables like ResortComments were enhanced to handle additional user interactions, such as detailed comments and ratings.
- **Describe future work that you think, other than the interface, that the application can improve on**
 - Future Work for Improvement:
 - Better Recommendations: Add personalized suggestions for resorts, lessons, and instructors based on user preferences and past activity.
 - Data Insights: Show trends like popular resorts, lesson demand, and average ratings over time.
 - Notifications: Notify users about schedule updates, resort conditions, or reminders for lessons.
- **Describe the final division of labor and how well you managed teamwork.**
 - Jing Li (Database Design and Implementation):
 - Designed and implemented the database schema, including the Lessons, Teach, InstructorComments, and ResortComments tables.
 - Developed and tested SQL triggers for updating average ratings dynamically when comments were added, updated, or deleted.
 - Ensured data integrity and handled complex relationships between tables like many-to-many mappings (e.g., instructors and lessons).
 - Ziqi Liu (Database Testing and Optimization):
 - Focused on testing the database design for edge cases, such as handling null values and recalculating averages correctly.
 - Optimized SQL queries for performance, especially for triggers and advanced stored procedures like filtering resorts by slopes and snowfall.
 - Debugged foreign key constraints and ensured smooth integration of new features into the database.
 - Yutong Chen (Backend Development):

- Built the backend API using Python Flask to interact with the database and handle user requests.
- Implemented endpoints for managing user actions, such as adding comments, enrolling in lessons, and filtering resorts.
- Worked on integrating the backend with frontend features.
- Yu-Tao Sun (Frontend and Backend Development):
 - Designed the user interface for the application, ensuring a seamless user experience.
 - Implemented pages for browsing resorts, viewing lessons, and providing ratings for instructors and resorts.
 - Focused on error handling and improving UI responsiveness for real-time interactions with the backend.
 - Help implement backend APIs to retrieve data from the database.
- Teamwork Management: Overall, teamwork was managed effectively through regular meetings, clear communication, and defined roles for each member. Each team member contributed their unique strengths to the project, and tasks were divided based on individual expertise and interest.